

Cookbook: Proactive Failure Readiness

Cookbook: Proactive Failure Readiness

How to find out your runbooks are useless before 3am.

The Problem

Every team has the same experience. Production breaks. An on-call engineer gets paged. They open Sentry and see `HTTP::TimeoutError: rate limit exceeded`. No breadcrumbs, no provider name, no customer ID. They spend 40 minutes digging through logs before they even understand what happened. There's no runbook, or the runbook is six months stale and references infrastructure that was replaced two quarters ago.

The fixes are obvious in hindsight: better Sentry context, up-to-date runbooks, tested mitigation steps. But nobody does it proactively because it's tedious, it's not urgent, and there's always a feature to ship.

StupidClaw automates the tedious part.

The Workflow

Scan your code for exception paths
↓
Simulate what those exceptions look like as Sentry alerts
↓
Feed the alerts into the ops pipeline
↓
Cluster related failures
↓
Score the instrumentation quality
↓
Draft runbooks (or find existing ones are stale)
↓
Test the runbooks against staging
↓
Fix the alerts that are too thin to debug
↓
Repeat — each cycle produces richer alerts and better runbooks

This isn't reactive incident response. This is a drill you run on a Tuesday afternoon so that when the real incident hits, the runbook already exists, the alerts already have context, and the on-call engineer already knows what to check.

Worked Example: CRM Database Outage

Step 1: Find the Exception Paths

Pick a service. Look at where it can fail. You don't need StupidClaw for this step — a code review, a `grep` for `rescue`, or a static analysis tool will do.

For a Rails CRM service talking to Postgres, the obvious paths are:

- **Connection pool exhaustion** — `ActiveRecord::ConnectionTimeoutError` when all connections are checked out and the timeout expires
- **Database unreachable** — `PG::ConnectionBad` when Postgres refuses connections entirely
- **Nil references in dependent services** — `NoMethodError` when a lookup returns nil because the data source is down
- **External API rate limits** — `HTTP::TimeoutError` (or whatever your HTTP client throws) when a third-party API throttles you

These aren't hypothetical. These are the errors that will fire eventually. The question is whether your alerts and runbooks are ready.

Step 2: Simulate the Alerts

Build Sentry event fixtures that look like what your monitoring would actually produce. Be honest about how much context your current instrumentation provides.

Here's the pool timeout alert — what Sentry would actually capture today:

```
{
  "platform": "ruby",
  "level": "error",
  "server_name": "scribbl-crm-1",
  "environment": "staging",
  "tags": {
    "service": "crm-service",
    "deployment": "kamal",
    "region": "westus2"
  },
  "breadcrumbs": {
    "values": [
      { "category": "db", "message": "SELECT * FROM sessions WHERE token = ?" },
      { "category": "rails.request", "message": "POST /sessions" }
    ]
  },
  "request": {
    "url": "https://crm.staging.scribbl.test/sessions",
    "method": "POST"
  },
  "exception": {
    "values": [
      {
        "type": "ActiveRecord::ConnectionTimeoutError",
        "value": "could not obtain connection from pool within 5.000 seconds",
        "stacktrace": {
          "frames": [
            { "filename": "lib/connection_pool.rb", "lineno": 118, "function": "checkout" },
            { "filename": "app/controllers/sessions_controller.rb", "lineno": 42, "function": "create" }
          ]
        }
      }
    ]
  }
}
```

And here's the billing rate limit — deliberately thin:

```
{
  "platform": "ruby",
  "level": "error",
  "server_name": "scribbl-billing-1",
  "tags": { "service": "billing-service" },
  "exception": {
    "values": [
      {
        "type": "HTTP::TimeoutError",
        "value": "rate limit exceeded",
        "stacktrace": {
          "frames": [
            { "filename": "app/services/payment_gateway.rb", "lineno": 88, "function": "charge" }
          ]
        }
      }
    ]
  }
}
```

No breadcrumbs. No provider name. No HTTP status code. No customer ID. This is what an on-call engineer would actually see at 3am — and it tells them almost nothing.

We built four fixtures total (pool timeout, connection refused, nil reference, rate limit) and saved them in `test/fixtures/sentry/`.

Step 3: Feed Them Into the Pipeline

Create a work item in the operations queue with the fixtures as input:

```
curl -X POST http://localhost:3000/api/v1/work_items \
-H "Content-Type: application/json" \
-d '{
  "queue": "operations",
  "title": "Failure readiness drill: CRM database outage",
  "spec_url": "test://crm-db-drill"
}'
```

Then run engine ticks. The pipeline takes it from here.

Step 4: Watch the Pipeline Work

The operations queue has six stages before human review. Each one uses a different Claude model matched to the complexity of the task:

Stage 1 — Ingest Signals (Haiku, ~50s)

Haiku normalizes the raw events. Extraction, not reasoning. It pulled out service names, error types, stack locations, and spotted the first pattern:

“Two database-related errors on crm-service (pool timeout and connection refused) — both targeting crm-postgres.internal in westus2, suggesting a correlated database availability pattern. Errors span two different hosts (scribble-crm-1, scribble-crm-2), ruling out a single-host network issue.”

Stage 2 — Cluster Failures (Sonnet, ~40s)

Sonnet grouped four alerts into three clusters:

Cluster	Severity	Alerts	Reasoning
crm-postgres-unavailable	high	pool timeout + connection refused	Same DB host, different app servers. Pool exhaustion is a downstream symptom of connection refusal.
notification-nil-reference	medium	nil reference	Different service, no proven shared dependency. But raised the hypothesis: “if notification-service queries crm-service’s database and that database is down, the lookup could return nil.”
billing-rate-limit	low	rate limit	Independent failure. External API throttle. No infrastructure overlap.

The clustering isn’t just grouping by service name. It reasoned about causal relationships — pool exhaustion is a *consequence* of connection refusal, not a separate failure.

Stage 3 — Assess Instrumentation (Sonnet, ~85s)

This is the step that makes the drill worthwhile. Sonnet scored each cluster’s instrumentation quality on five dimensions (error specificity, context richness, breadcrumbs, reproducibility, structured metadata):

Cluster	Score (out of 5)	Verdict
crm-postgres-unavailable	2.2	thin
notification-nil-reference	2.0	thin
billing-rate-limit	1.2	thin

Every cluster failed. On the billing alert:

“Error message is ‘rate limit exceeded’ with no detail on which payment provider, API endpoint, or rate limit tier was hit. HTTP::TimeoutError is a misleading exception type for a rate limit. No request context: no customer/invoice/payment ID, no amount, no idempotency key. Zero reproducibility context.”

Then it spawned three work items into the **development** queue — specific

instrumentation fixes for each service, with inline specs detailing exactly which Sentry.set_context calls to add, which tags to set, and where to add breadcrumbs.

Stage 4 — Map Runbooks (Sonnet, ~55s)

Sonnet searched the repository for existing runbooks. Found none. All three clusters mapped to “missing.” It noted what each runbook would need to cover and the expected file convention (services/<name>/docs/runbooks/*.yaml).

Stage 5 — Draft Runbooks (Opus, ~150s)

Opus synthesized everything upstream and drafted three complete runbooks. Not checklists — actual operational procedures with runnable commands:

For the Postgres outage runbook: - **Observe:** pg_isready -h crm-postgres.internal -p 5432, SELECT count(*), state FROM pg_stat_activity GROUP BY state, Rails console pool stats - **Mitigate:** Restart Postgres if down. Terminate idle-in-transaction sessions >5 minutes (exact SQL provided). Rolling restart via kamal app boot if pool won't drain. - **Verify:** Manual POST /sessions test, monitor Sentry 15 minutes, confirm pool stats show idle connections - **Escalation:** Postgres won't restart → DBA. Connections maxed by active queries → DBA (don't kill active queries without approval). Network partition → infra team. 30+ minutes → incident commander.

For the billing rate limit runbook, it caught the error type problem:

```
“HTTP::TimeoutError is a misleading exception type for a rate limit — this is likely an HTTP 429 response, not a network timeout.”
```

And warned about the retry storm risk:

```
“If billing jobs retry on failure without exponential backoff, they can amplify the rate limit by retrying immediately.”
```

Stage 6 — Human Review (gate)

The pipeline stopped. It will not push runbooks to staging or production without human approval. This is the gate where an engineer reviews the drafted runbooks, edits them, and decides whether to proceed.

Step 5: Evaluate the Runbooks

This is where the drill pays off. You now have three runbooks that were drafted against specific failure scenarios. Read them critically:

- **Are the observe commands correct?** Can you actually run pg_isready from an app server? Do you have the readonly Postgres user it assumes?
- **Are the mitigation steps safe?** The runbook says “restart Postgres if confirmed down” — is that actually safe in your environment? Who has access?
- **Does the escalation matrix match your org?** Does your team have a “DBA” role? Is “incident commander” a thing?
- **What's missing?** The runbook doesn't know about your Datadog dashboards, your PagerDuty rotation, or your Slack incident channel.

The runbooks are drafts, not gospel. The value is that they exist at all, they're structured consistently, and they're based on the actual alerts your monitoring would produce — not a hypothetical scenario someone wrote on a wiki two years ago.

Step 6: Fix the Thin Alerts

The pipeline found that every alert was too thin to debug effectively. The spawned development work items have specific fixes:

Before: billing-service alert

```
{
  "type": "HTTP::TimeoutError",
  "value": "rate limit exceeded"
}
```

An on-call engineer sees this and starts a 40-minute investigation.

After: billing-service alert (with instrumentation fixes applied)

```
{
  "type": "BillingService::RateLimitError",
  "value": "Stripe rate limit: 429 (Retry-After: 30s)",
  "tags": {
    "provider": "stripe",
    "http_status": "429",
    "rate_limit_tier": "standard"
  },
  "contexts": {
    "payment": {
      "customer_id": "cus_abc123",
      "invoice_id": "inv_xyz789",
      "amount_cents": 4999,
      "idempotency_key": "charge_20260505_inv_xyz789"
    }
  },
  "breadcrumbs": [
    { "message": "POST /v1/charges (attempt 1) → 200 OK" },
    { "message": "POST /v1/charges (attempt 2) → 200 OK" },
    { "message": "POST /v1/charges (attempt 3) → 429 Rate Limited" }
  ]
}
```

Now the engineer sees: it's Stripe, it's a 429, Retry-After is 30 seconds, it happened on the third attempt, and here's the customer and invoice. The runbook can reference specific fields because the alert actually contains them.

Before: crm-service pool timeout

```
{
  "type": "ActiveRecord::ConnectionTimeoutError",
  "value": "could not obtain connection from pool within 5.000 seconds"
}
```

After: crm-service pool timeout (with instrumentation fixes applied)

```
{
  "type": "ActiveRecord::ConnectionTimeoutError",
  "value": "could not obtain connection from pool within 5.000 seconds",
  "tags": {
    "request_id": "req_7f3a2b",
    "tenant_id": "tenant_42",
    "database_host": "crm-postgres.internal"
  },
  "contexts": {
    "database": {
      "pool_size": 10,
      "checked_out": 10,
      "idle": 0,
      "waiting": 47,
      "checkout_timeout": 5.0
    },
    "pg_stat": {
      "active_connections": 95,
      "max_connections": 100,
      "idle_in_transaction": 23
    }
  },
  "breadcrumbs": [
    { "message": "Connection checkout requested" },
    { "message": "Pool exhausted: 10/10 checked out, 47 waiting" },
    { "message": "Timeout after 5.0s waiting for connection" }
  ]
}
```

The runbook can now say “if `idle_in_transaction > 20`, terminate sessions older than 5 minutes” because the data is in the alert.

Step 7: Repeat

Apply the instrumentation fixes. Rebuild the fixtures to reflect the richer alerts. Run the pipeline again. The instrumentation scores should go up, the runbooks should be more specific, and the spawned dev work items should decrease (or disappear).

This is the loop:

Agents don't self-certify. When an agent finishes:

1. It produces a report and artifacts (self-report)
2. The engine runs predicates against the artifacts (independent verification)

If the `clusters_created` predicate doesn't find a `clusters` artifact with actual data, the stage fails — regardless of what the agent claims. We caught a real bug this way: the adapter was labeling artifacts with the wrong kind. The agents all said "done." The predicates all said "prove it." The system caught the mismatch.

Cost Funnel

Not every stage needs the most expensive model:

- **Haiku** for intake — extraction and normalization
- **Sonnet** for clustering, assessment, mapping — moderate reasoning
- **Opus** for runbook drafting — synthesis across all upstream context

Change one line in the YAML to swap models.

Cross-Queue Spawn

The ops pipeline can create work items in the dev pipeline. When `assess_instrumentation` finds thin alerts, it spawns dev tasks with specific inline specs. Different queue, different stages, different agents. The ops pipeline identifies the problem; the dev pipeline fixes it.

When to Run This

- **Before launching a new service** — simulate the failures it will have and make sure your monitoring and runbooks are ready
- **After a real incident** — rebuild the alerts from the post-mortem and verify your runbooks would have caught it
- **Quarterly** — re-scan code for new exception paths, rebuild fixtures, check for stale runbooks
- **After major dependency changes** — new database, new payment provider, new third-party API — simulate the failure modes before they surprise you